

رسالة محمد

# یادگیری ماشین

مدرس: محمدرضا محمدی

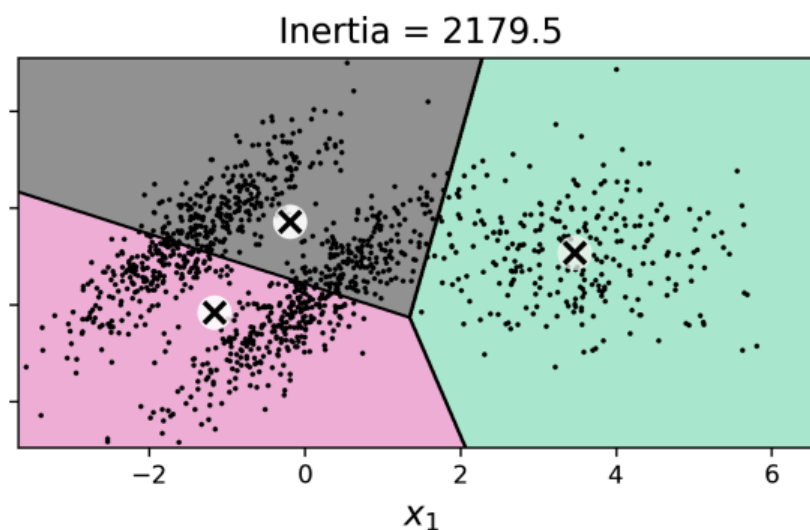
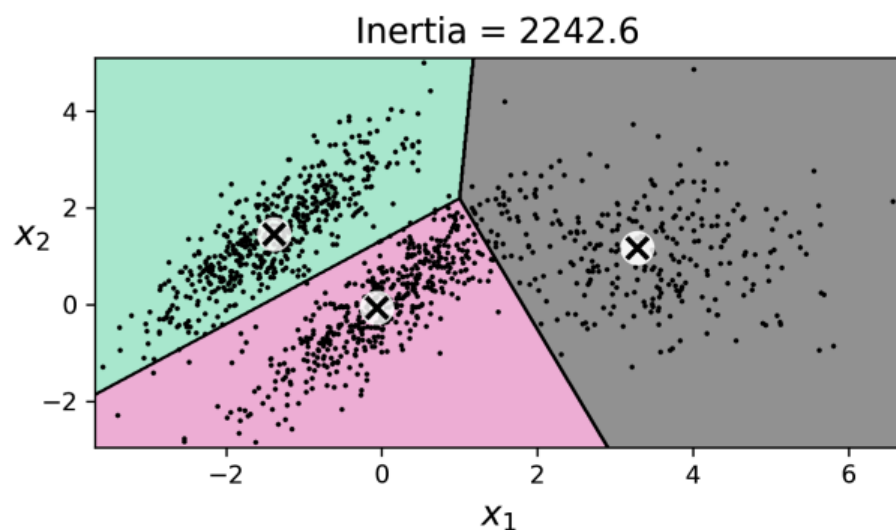
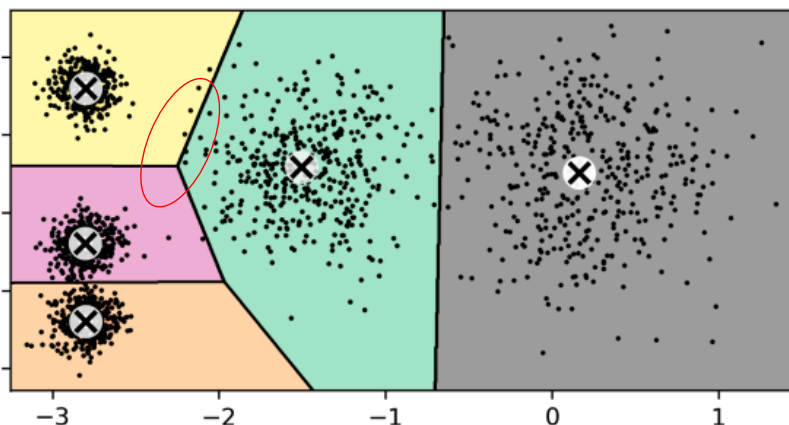
بهار ۱۴۰۲

# الگوریتم‌های خوشه‌بندی

## Clustering Algorithms

# محدودیت‌های k-means

- حساسیت به مراکز اولیه و نیاز به تعیین تعداد خوشه‌ها
- اگر خوشه‌ها دارای اندازه‌های متفاوتی باشند، می‌تواند دچار چالش شود
- تاثیرگذاری اختلاف در چگالی خوشه‌ها
- اگر شکل خوشه‌ها غیرکروی باشند، نمی‌تواند عملکرد مطلوبی داشته باشد



# ناحیه‌بندی تصویر با استفاده از خوشه‌بندی

- در ناحیه‌بندی تصویر، یک تصویر به چندین قسمت تقسیم می‌شود
  - در ناحیه‌بندی رنگی (Color Segmentation)، پیکسل‌های دارای رنگ مشابه به یک ناحیه اختصاص می‌یابند
  - در ناحیه‌بندی معنایی (Semantic Segmentation)، پیکسل‌های مربوط به بخش‌های مختلف یک شیء به یک ناحیه اختصاص می‌یابند
- با استفاده از معماری‌های پیچیده مانند شبکه‌های عصبی کانولوشنی قابل انجام است





```
>>> import PIL
```

```
>>> image = np.asarray(PIL.Image.open(filepath))
```

```
>>> image.shape
```

```
(533, 800, 3)
```

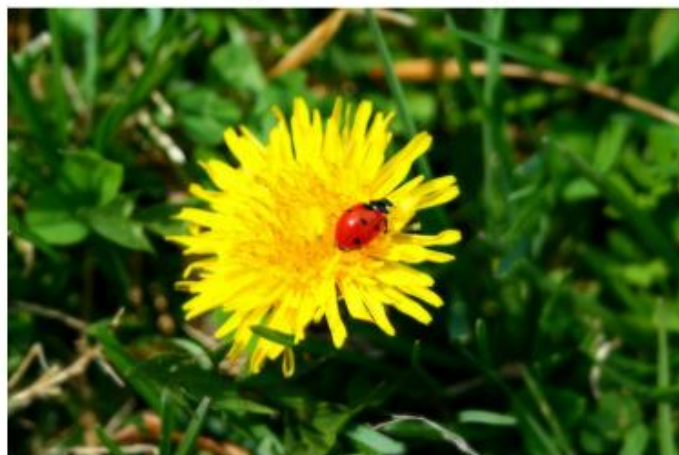
```
X = image.reshape(-1, 3)
```

```
kmeans = KMeans(n_clusters=8, random_state=42).fit(X)
```

```
segmented_img = kmeans.cluster_centers_[kmeans.labels_]
```

```
segmented_img = segmented_img.reshape(image.shape)
```

Original image



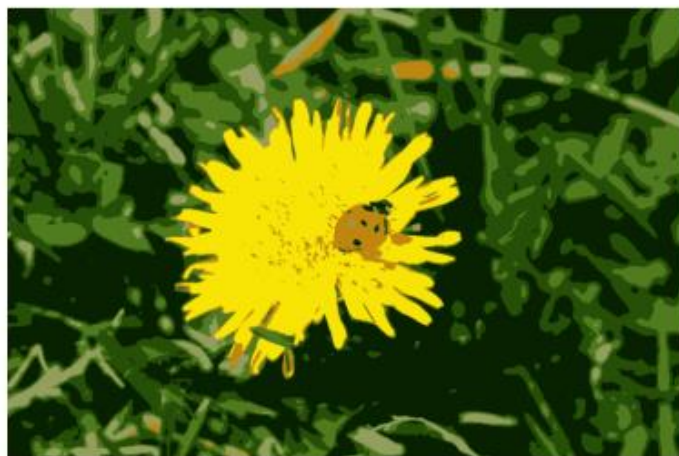
10 colors



8 colors



6 colors



4 colors



2 colors



# یادگیری نیمه نظارتی با استفاده از خوشه‌بندی

- فرض کنید می‌خواهیم از یک مجموعه داده شامل 1,797 تصویر  $8 \times 8$  برای شناسایی رقم‌های 0 تا 9 استفاده کنیم

```
from sklearn.datasets import load_digits
```

```
X_digits, y_digits = load_digits(return_X_y=True)
X_train, y_train = X_digits[:1400], y_digits[:1400]
X_test, y_test = X_digits[1400:], y_digits[1400:]
```

- فقط از برچسب ۵۰ نمونه استفاده می‌کنیم

```
from sklearn.linear_model import LogisticRegression
```

```
n_labeled = 50
log_reg = LogisticRegression(max_iter=10_000)
log_reg.fit(X_train[:n_labeled], y_train[:n_labeled])
```

```
>>> log_reg.score(X_test, y_test)
0.7481108312342569
```

- اگر از تمام نمونه‌های آموزشی استفاده شود، دقت 90.7% می‌شود

# یادگیری نیمه نظارتی با استفاده از خوشه بندی

- در یادگیری نیمه نظارتی، ابتدا تمام نمونه های آموزشی را به ۵۰ خوشه تقسیم می کنیم

```
k = 50  
kmeans = KMeans(n_clusters=k, random_state=42)  
X_digits_dist = kmeans.fit_transform(X_train)  
representative_digit_idx = np.argmin(X_digits_dist, axis=0)  
X_representative_digits = X_train[representative_digit_idx]
```

- نزدیک ترین نمونه ها به مراکز خوشه ها:

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 3 | 6 | 0 | 7 | 9 | 2 | 4 | 8 | 9 |
| 5 | 4 | 7 | 1 | 2 | 6 | 1 | 2 | 5 | 1 |
| 4 | 1 | 3 | 3 | 8 | 8 | 2 | 5 | 6 | 9 |
| 1 | 4 | 0 | 6 | 8 | 3 | 4 | 6 | 7 | 2 |
| 4 | 1 | 0 | 7 | 5 | 1 | 9 | 9 | 3 | 7 |



# یادگیری نیمه نظارتی با استفاده از خوشه بندی

- حال این ۵۰ نمونه را به صورت دستی برچسب می زنیم و مدل را آموزش می دهیم

```
y_representative_digits = np.array([1, 3, 6, 0, 7, 9, 2, ..., 5, 1, 9, 9, 3, 7])
```

```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_representative_digits, y_representative_digits)
>>> log_reg.score(X_test, y_test)
0.8488664987405542
```

|   |   |   |   |   |   |   |   |   |   |
|---|---|---|---|---|---|---|---|---|---|
| 1 | 3 | 6 | 0 | 7 | 9 | 2 | 4 | 8 | 9 |
| 5 | 4 | 7 | 1 | 2 | 6 | 1 | 2 | 5 | 1 |
| 4 | 1 | 3 | 3 | 8 | 8 | 2 | 5 | 6 | 9 |
| 1 | 4 | 0 | 6 | 8 | 3 | 4 | 6 | 7 | 2 |
| 4 | 1 | 0 | 7 | 5 | 1 | 9 | 9 | 3 | 7 |

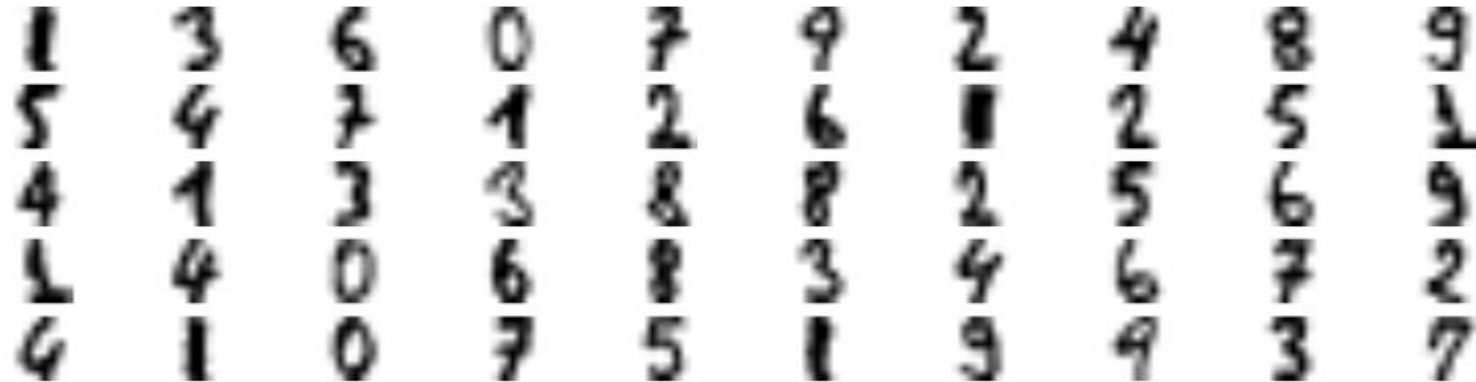
# یادگیری نیمه نظارتی با استفاده از خوشه‌بندی

- می‌توان برچسب هر کدام از این نمونه‌های نماینده (Representative) را به تمام نمونه‌های خوشه مربوطه اختصاص داد

- به این کار انتشار برچسب (Label Propagation) گفته می‌شود

```
y_train_propagated = np.empty(len(X_train), dtype=np.int64)
for i in range(k):
    y_train_propagated[kmeans.labels_ == i] = y_representative_digits[i]
```

```
>>> log_reg = LogisticRegression()
>>> log_reg.fit(X_train, y_train_propagated)
>>> log_reg.score(X_test, y_test)
0.8942065491183879
```



# یادگیری نیمه نظارتی با استفاده از خوشه‌بندی

- می‌توان از نمونه‌هایی که نسبت به مرکز خوشه خود فاصله زیادی دارند و احتمال دارد نمونه پرت باشند صرف نظر کرد

```
X_cluster_dist = X_digits_dist[np.arange(len(X_train)), kmeans.labels_]
for i in range(k):
    in_cluster = (kmeans.labels_ == i)
    cluster_dist = X_cluster_dist[in_cluster]
    cutoff_distance = np.percentile(cluster_dist, percentile_closest)
    above_cutoff = (X_cluster_dist[in_cluster] > cutoff_distance)
    X_cluster_dist[in_cluster & above_cutoff] = -1
```

```
partially_propagated = (X_cluster_dist != -1)
X_train_partially_propagated = X_train[partially_propagated]
y_train_partially_propagated = y_train[partially_propagated]
```

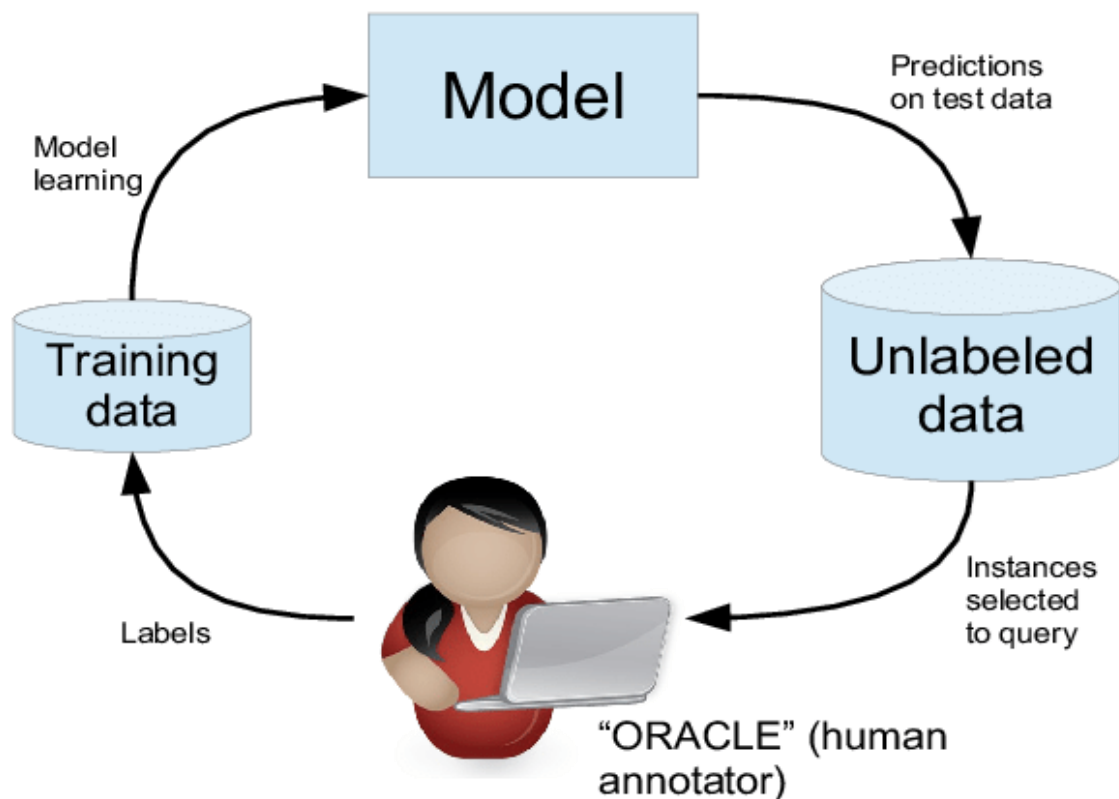
```
>>> log_reg = LogisticRegression(max_iter=10_000)
>>> log_reg.fit(X_train_partially_propagated, y_train_partially_propagated)
>>> log_reg.score(X_test, y_test)
0.9093198992443325
```

- از دقت مدل با نظارت کامل (90.7%) هم بیشتر شد!

- به دلیل حذف برخی از داده‌های پرت
- 97.5% از برچسب‌های انتشار یافته درست بوده‌اند

# یادگیری فعال (Active Learning)

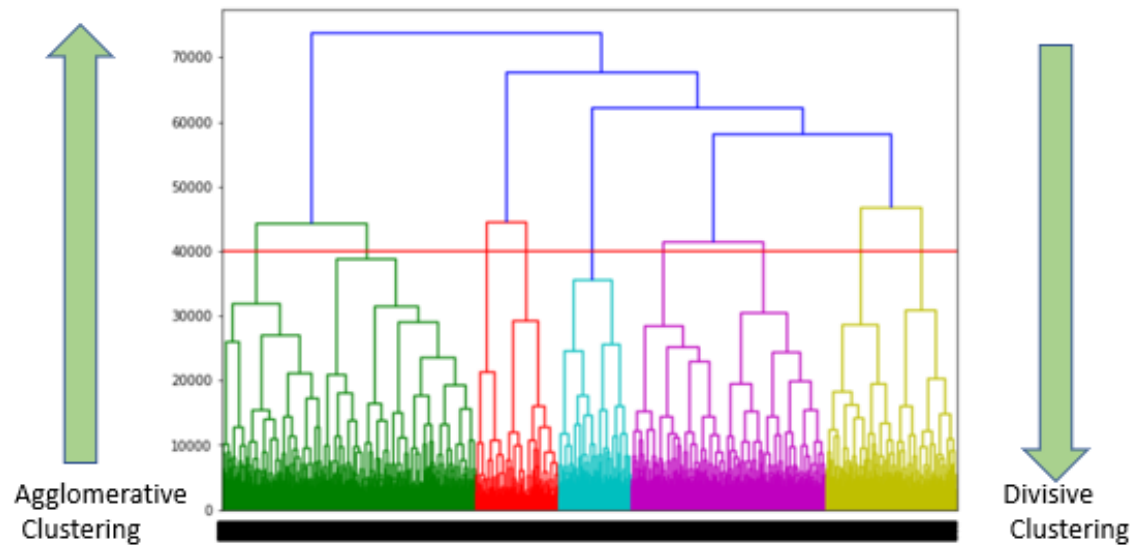
- در یادگیری فعال، با کمک گرفتن از یک خبره، برچسب گذاری برخی نمونه‌های بدون برچسب انجام شده و مدل به روزرسانی می‌شود



- مدل بر روی نمونه‌های برچسب خورده موجود آموزش داده می‌شود و بر روی نمونه‌های بدون برچسب پیش‌بینی را انجام می‌دهد
- نمونه‌هایی که بیشترین عدم قطعیت را دارند برای برچسب گذاری به خبره داده می‌شوند
- این فرآیند تکرار می‌شود تا بهبود عملکرد مدل متوقف شود

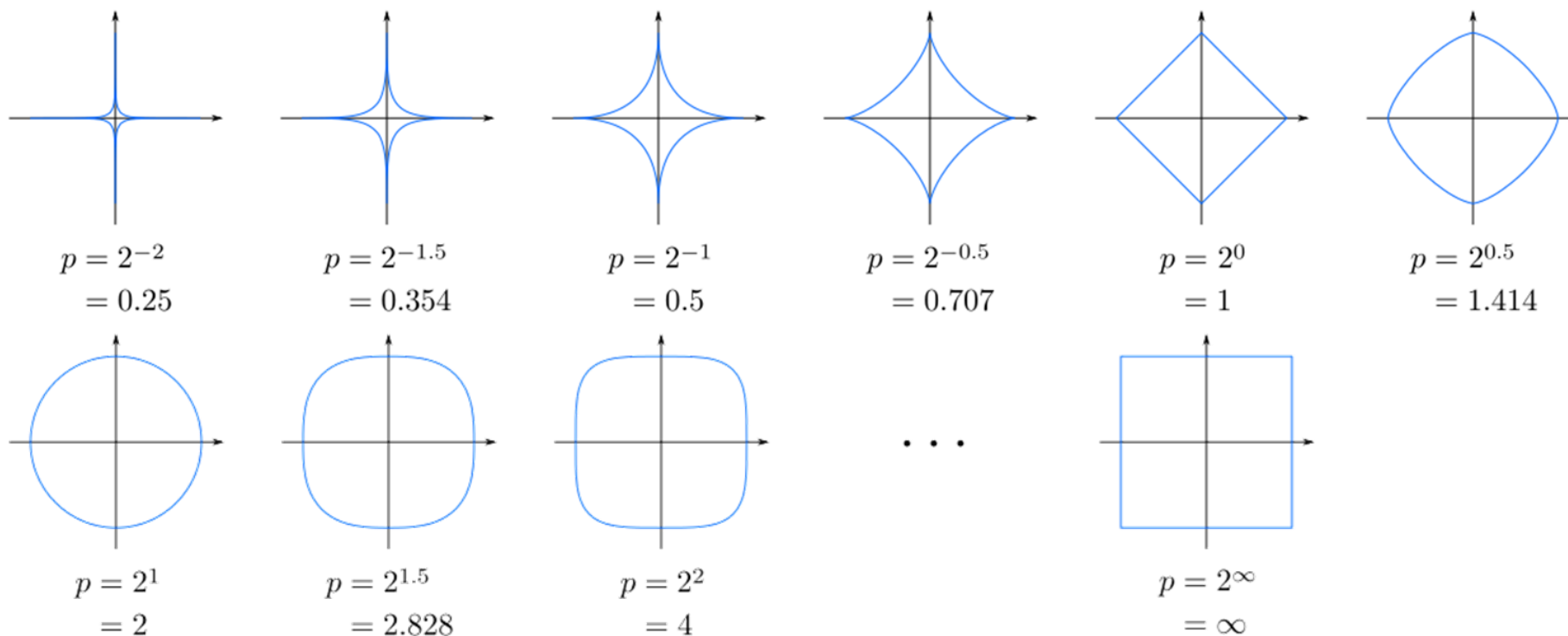
# خوشه‌بندی سلسله‌مراتبی (Hierarchical)

- در رویکرد تجمعی (agglomerative)، ابتدا هر نمونه به عنوان یک خوشه در نظر گرفته می‌شود و سپس به صورت تکرارشونده، دو خوشه‌ای که کمترین فاصله را داشته باشند با هم تجمیع می‌شوند
- در رویکرد تقسیم‌شونده (divisive)، با یک خوشه که شامل تمام نمونه‌ها است شروع می‌شود و خوشه‌های دارای واریانس زیاد به خوشه‌های کوچکتر تقسیم می‌شوند



# خوشه‌بندی تجمعی

- ابتدا باید معیاری برای محاسبه فاصله نمونه‌ها از هم مشخص کرد
- فاصله اقلیدوسی و فاصله نرم  $p$  متداول‌ترین معیارهای محاسبه فاصله دو نمونه هستند





# خوشه‌بندی تجمعی

• سپس باید فاصله میان دو خوشه را محاسبه کرد که متداول‌ترین آنها عبارتند از:

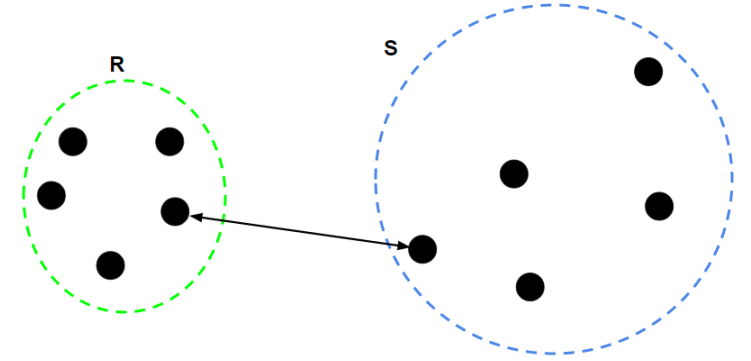
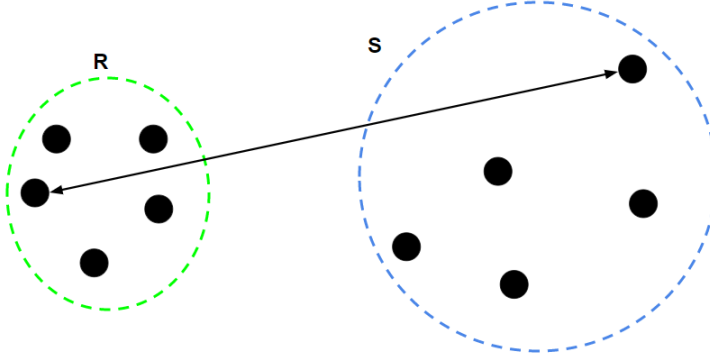
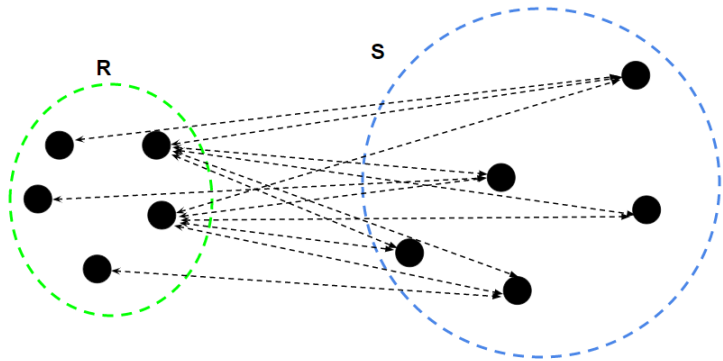
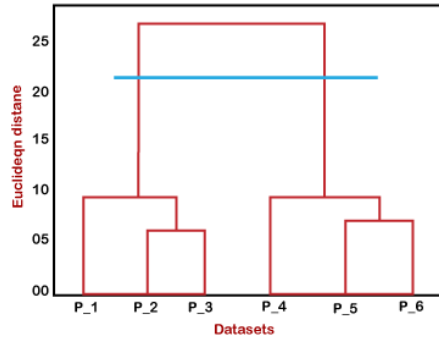
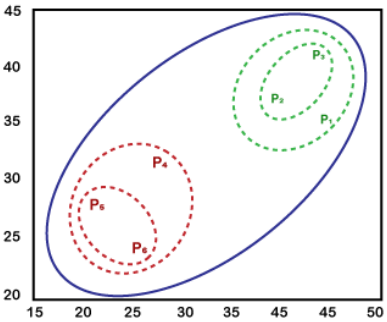
- single: کمترین فاصله بین نمونه‌های دو خوشه

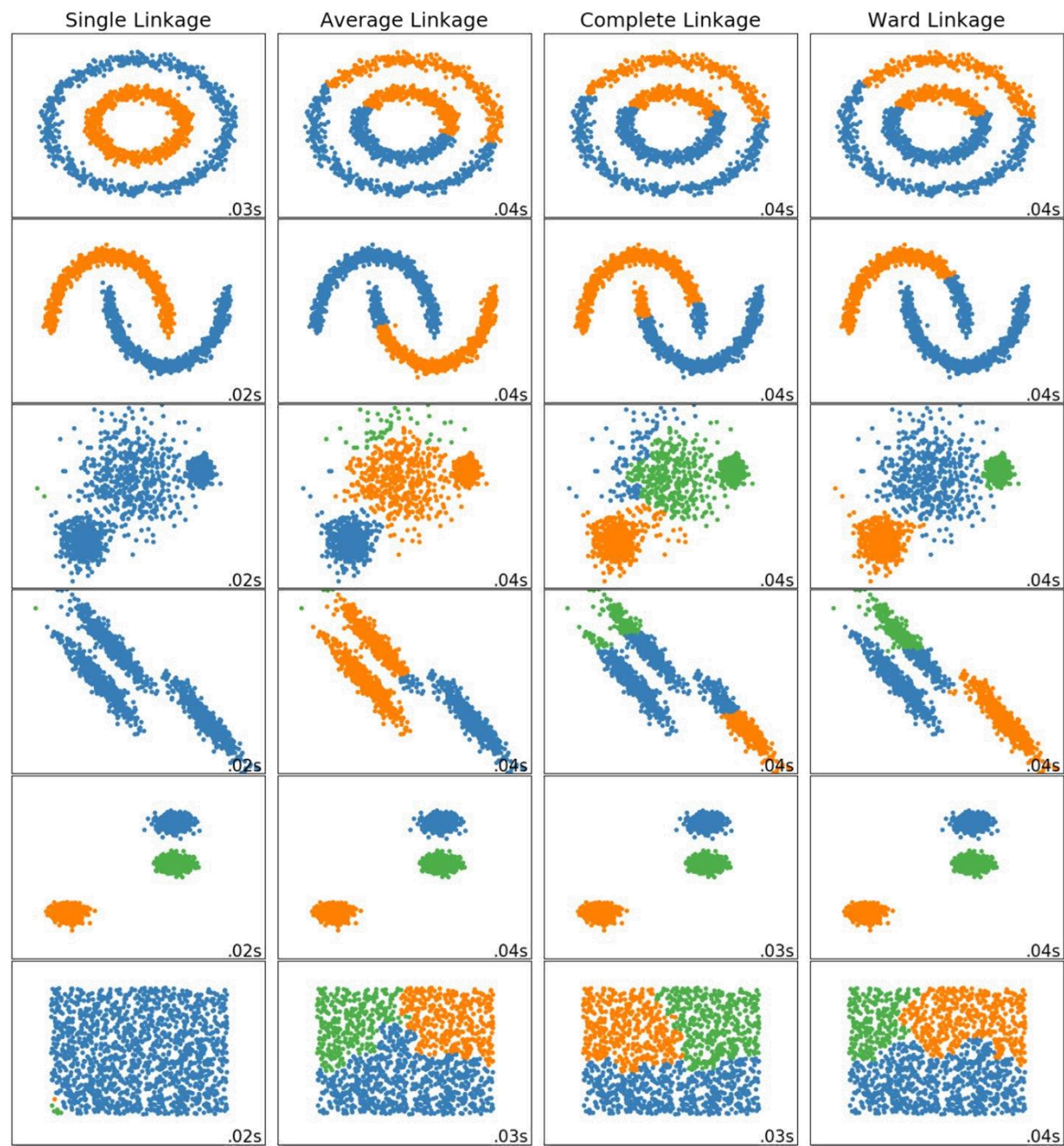
- complete: بیشترین فاصله بین نمونه‌های دو خوشه

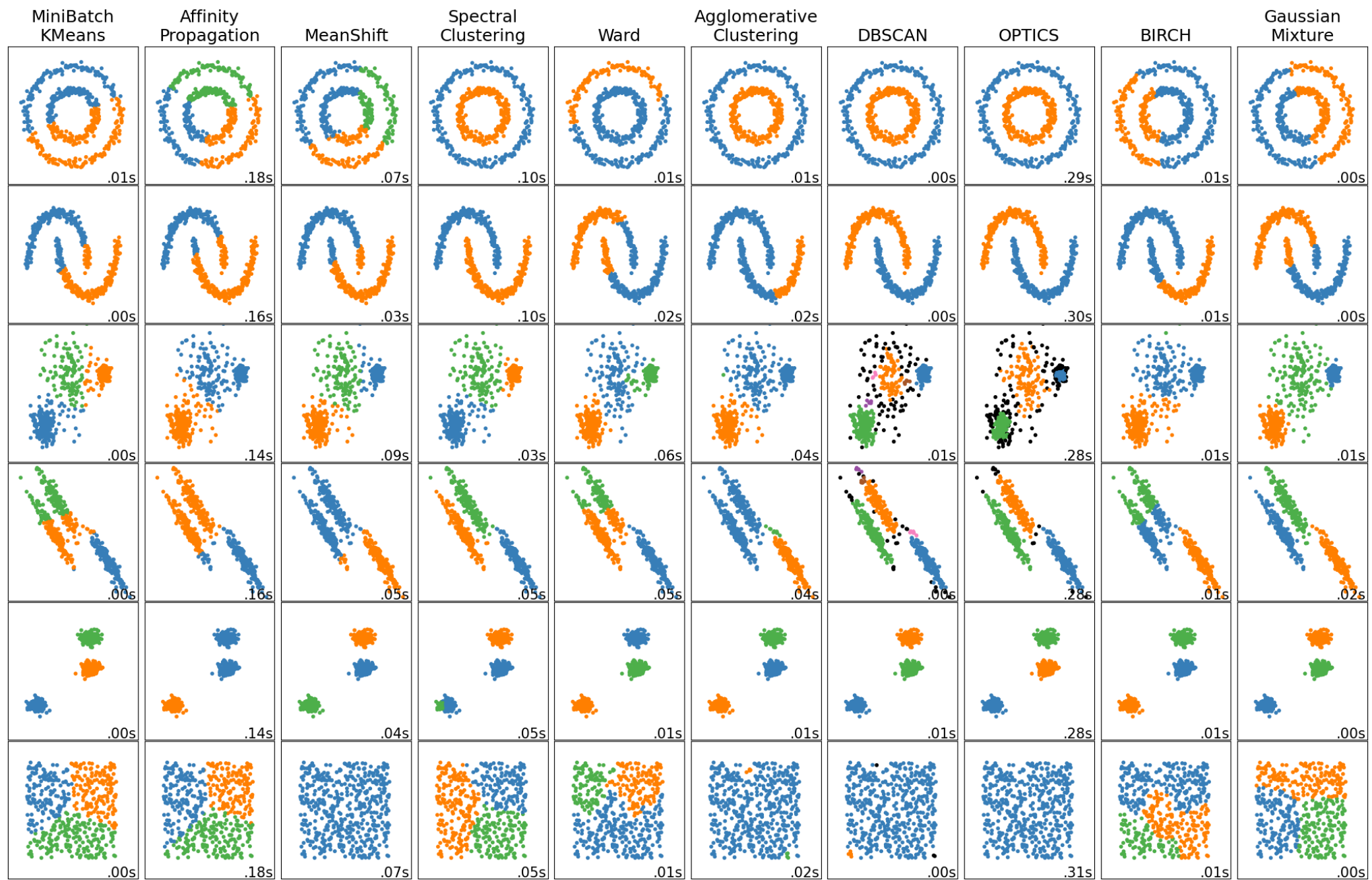
- average: میانگین فاصله نمونه‌های دو خوشه یا فاصله مراکز دو خوشه

- ward: واریانس خوشه تجمیع شده

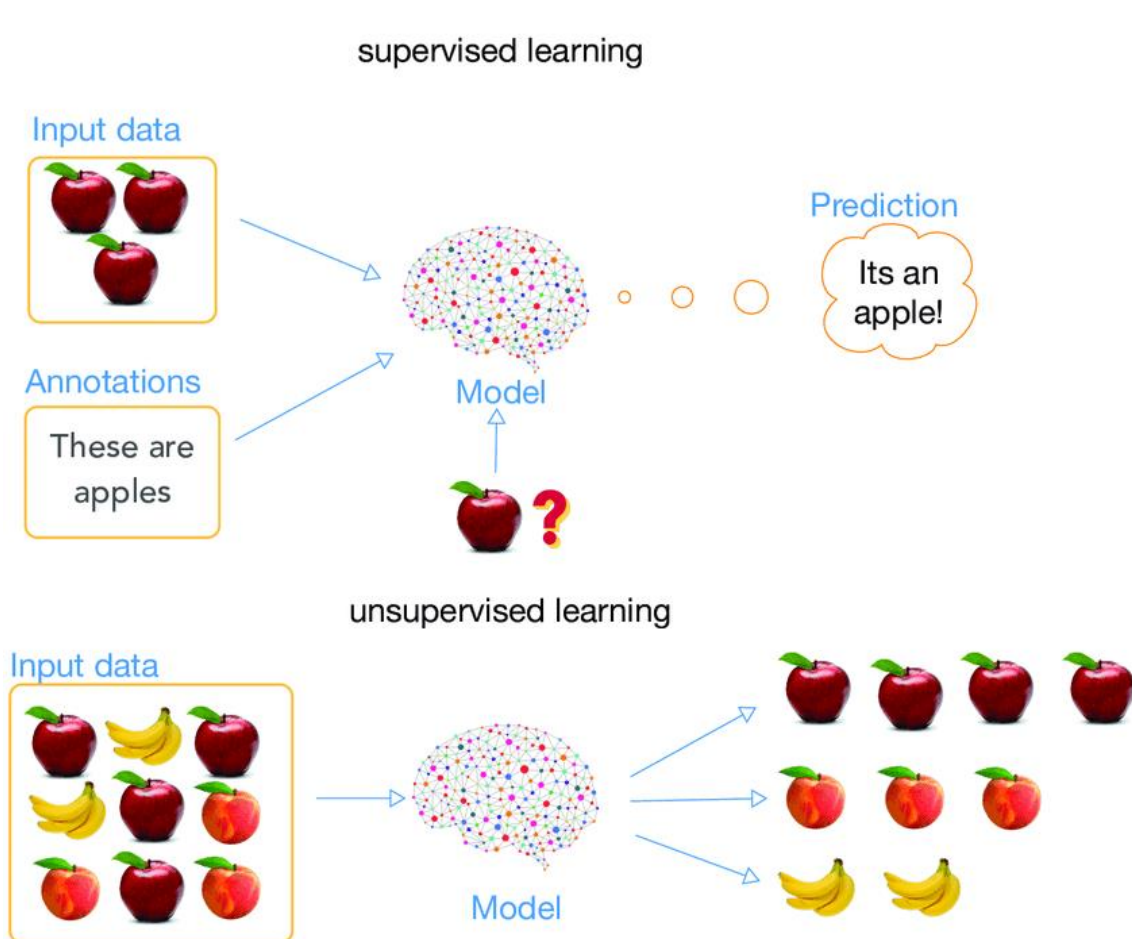
• برای توقف، می‌توان تعداد خوشه یا حداکثر فاصله را مشخص کرد







# معیارهای ارزیابی الگوریتم‌های خوشه‌بندی



- در ارزیابی الگوریتم‌های خوشه‌بندی، می‌توان از مجموعه داده‌های دارای برچسب استفاده کرد
- خوشه‌بندی به صورت بدون نظارت انجام می‌شود و در این گام هیچ استفاده‌ای از برچسب‌ها نمی‌شود
- اما برای ارزیابی از برچسب‌ها استفاده می‌شود و حالت ایده‌آل این است که تمام نمونه‌های یک کلاس در یک خوشه خالص قرار داشته باشند

# (Normalized Mutual Information) NMI

- اطلاعات متقابل برای دو توزیع به صورت زیر تعریف می شود:

$$MI = H(Y) - H(Y|C)$$

$$H(Y) = - \sum_{i=1}^n P(y_i) \log(P(y_i))$$

- $H(Y)$  آنترپی برچسب های کلاس صحیح هستند:

-  $P(y_i)$  درصد برچسب های متعلق به کلاس  $i$ ام است

- $H(Y|C)$  آنترپی شرطی برچسب های کلاس صحیح به شرط نتایج خوشه بندی است

$$H(Y|c_j) = - \sum_{i=1}^n P(y_i|c_j) \log(P(y_i|c_j))$$

$$H(Y|C) = \sum_{j=1}^c P(c_j) H(Y|c_j)$$



# (Normalized Mutual Information) NMI

$$MI = H(Y) - H(Y|C)$$

$$H(Y) = - \sum_{i=1}^n P(y_i) \log(P(y_i))$$

$$H(Y|c_j) = - \sum_{i=1}^n P(y_i|c_j) \log(P(y_i|c_j))$$

$$H(Y|C) = \sum_{j=1}^c P(c_j) H(Y|c_j)$$

$$H(C) = - \sum_{i=1}^c P(c_i) \log(P(c_i))$$

$$NMI = \frac{2MI}{H(Y) + H(C)}$$

- معیار MI نرمالیزه نیست، برای مقایسه بهتر، آن را به گونه‌ای نرمالیزه می‌کنیم که بین ۰ و ۱ باشد

- اگر خوشه‌بندی کاملاً دقیق باشد

$$H(Y) = H(C) \quad -$$

$$H(Y|c_j) = 0 \quad -$$

$$H(Y|C) = 0 \quad -$$

$$MI = H(Y) \quad -$$

$$NMI = 1 \quad -$$



# (Normalized Mutual Information) NMI

$$MI = H(Y) - H(Y|C)$$

$$H(Y) = - \sum_{i=1}^n P(y_i) \log(P(y_i))$$

$$H(Y|c_j) = - \sum_{i=1}^n P(y_i|c_j) \log(P(y_i|c_j))$$

$$H(Y|C) = \sum_{j=1}^c P(c_j) H(Y|c_j)$$

$$H(C) = - \sum_{i=1}^c P(c_i) \log(P(c_i))$$

$$NMI = \frac{2MI}{H(Y) + H(C)}$$

```
from sklearn.metrics.cluster import normalized_mutual_info_score
normalized_mutual_info_score([0, 0, 1, 1], [0, 0, 1, 1])
```

```
>>> 1.0
```

```
normalized_mutual_info_score([0, 0, 1, 1], [1, 1, 0, 0])
```

```
>>> 1.0
```

```
normalized_mutual_info_score([0, 0, 1, 1], [0, 1, 2, 3])
```

```
>>> 0.0
```

```
normalized_mutual_info_score([0, 0, 1, 1, 2, 2, 2], [2, 2, 0, 0, 1, 1, 1])
```

```
>>> 1.0
```

```
normalized_mutual_info_score([0, 0, 1, 1, 2, 2, 2], [2, 2, 0, 0, 1, 1, 3])
```

```
>>> 0.8878
```

```
normalized_mutual_info_score([0, 0, 1, 1, 2, 2, 2], [2, 3, 0, 0, 1, 1, 3])
```

```
>>> 0.7248
```